

# CHƯƠNG TRÌNH ĐÀO TẠO **NEXTJS - 3**

**HRM.L&D. 2026**

1

Xử lý Data Fetching trong NextJS

2

NextJS Server Side&Client Side Rendering

3

Advanced Routing trong NextJS

4

Sử dụng các hàm mới trong Data Fetching

5

Thực hành luyện tập

# NextJS Data Fetching

- Next.js hỗ trợ nhiều phương pháp data-fetching, chẳng hạn như static generation (SSG), server-side rendering (SSR), và client-side fetching. Next.js bảo đảm cơ chế bảo mật bằng cách truy xuất dữ liệu phía server và sau đó gửi dữ liệu về phía máy khách theo yêu cầu. Cơ chế server-side data fetching cung cấp hiệu suất tốt hơn và hỗ trợ kết quả SEO tốt hơn bằng cách giảm bớt khối lượng mã JS phía máy khách.
- Next.js cung cấp 3 kiểu phương pháp data fetching như sau:
  - Server-side Fetching
  - Client-side Fetching
  - Incremental Static Regeneration

# NextJS Data Fetching

- Trong server-side data fetching, dữ liệu được truy xuất ở phía server và sau đó được chuyển lại phía máy khách. Có 2 kiểu như sau
  - Static Site Generation
    - Là hành vi mặc định trong Next.js. Trong phương pháp này, Next.js sẽ tạo ra các pages tại thời điểm xây dựng (build time) và sử dụng lại các trang này đối với mỗi request. Cơ chế này được khuyến nghị đối với các trang không yêu cầu cập nhật thường xuyên.
  - Server-side Rendering
    - Trong phương pháp này, dữ liệu sẽ được truy vấn đối với mỗi request lên server. Phương pháp này được khuyến nghị đối với các trang có yêu cầu cập nhật dữ liệu thường xuyên.

# NextJS Data Fetching

- Ví dụ về Static Site Generation

```
export async function getStaticProps() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/1');
  const post = await res.json();

  return {
    props: { post },
  };
}

export default function Post({ post }) {
  return (
    <div>
      <h1>{post.title}</h1>
      <p>{post.body}</p>
    </div>
  );
}
```

# NextJS Data Fetching

- Ví dụ về Server-side Rendering

```
export async function getServerSideProps() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/1');
  const post = await res.json();

  return {
    props: { post },
  };
}

export default function Post({ post }) {
  return (
    <div>
      <h1>{post.title}</h1>
      <p>{post.body}</p>
    </div>
  );
}
```

# NextJS Data Fetching

- Client-side fetching
  - Trong phương pháp này, quá trình truy xuất dữ liệu sẽ diễn ra tại trình duyệt ở phía người dùng. Phương pháp này nên cân nhắc vì nó sẽ ảnh hưởng đến hiệu suất và kết quả SEO của website.
- Ví dụ:

```
"use client";

import { useEffect, useState } from 'react';

export default function Post() {
  const [post, setPost] = useState(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts/1')
      .then(res => res.json())
      .then(data => setPost(data));
  }, []);

  if (!post) return <p>Loading...</p>;

  return (
    <div>
      <h1>{post.title}</h1>
      <p>{post.body}</p>
    </div>
  );
}
```

# NextJS Data Fetching

- Incremental Static Regeneration
  - Là một đặc điểm trong Next.js trong đó các trang tĩnh có thể được cập nhật sau khi đã triển khai ứng dụng mà không cần thiết phải tiến hành full rebuild.
  - Quá trình này cho phép tạo các trang tĩnh và sau đó có thể cập nhật các trang này khi ứng dụng đang được thực thi.
- Ví dụ

```
import { useEffect } from 'react';
import { getServerSideProps } from 'next';

export async function getStaticProps() {
  const data = await fetchData(); // Fetch data from API

  return {
    props: { data },
    revalidate: 10, // Regenerate the page every 10 seconds
  };
}

export default function Home({ data }) {
  return (
    <div>
      <h1>Home Page</h1>
      <p>This is the home page.</p>
      <div>{JSON.stringify(data)}</div>
    </div>
  );
}
```

# NextJS Server Side Rendering

- Server-Side Rendering (SSR) là một đặc điểm trong Next.js, trong đó server sẽ xử lý request và gửi lại toàn bộ trang HTML về cho client. Trong cơ chế Server-Side Rendering, trang HTML sẽ được tạo ra trên mỗi request, do đó trình duyệt sẽ không phải đợi JavaScript thực thi trước khi hiển thị nội dung. Điều này sẽ giúp load trang nhanh hơn và hiện hữu đối với các search engines.
- Có thể nói rằng, Server-Side Rendering sẽ đảm bảo nội dung động của trang web sẽ được truy xuất và được sinh ra ở trên server trước khi được gửi cho client.

# NextJS Server Side Rendering

- Cơ chế làm việc của server side rendering
  - Mỗi khi người dùng requests một trang, server sẽ thực hiện các việc sau:
    - Truy xuất dữ liệu cần thiết cho trang.
    - Sinh ra nội dung HTML động.
    - Gửi lại trang HTML về trình duyệt.
- Ví dụ:

```
export async function getServerSideProps() {
  // Fetch data from API
  const response = await fetch('https://link/to/api');
  const data = await response.json();

  // Pass data as prop to the page component
  return { props: { data } };
}

export default function Page({ data }) {
  return (
    <div>
      <h1>Server-Side Rendered Data</h1>
      <ul>
        {data.map((item) => (
          <li key={item.id}>{item.title}</li>
        ))}
      </ul>
    </div>
  )
}
```

# NextJS Client Side Rendering

- Client Side Rendering
  - Là một kỹ thuật được sử dụng trong các ứng dụng web applications, trong đó nội dung sẽ được rendered trong trình duyệt bằng cách sử dụng JavaScript. Trình duyệt sẽ downloads một trang HTML nhỏ nhất và JavaScript cần cho trang web, sau đó JavaScript sẽ được dùng để cập nhật DOM và render trang.
  - Khi ứng dụng được load lần đầu tiên, người dùng sẽ cảm thấy hơi delay trước khi họ có thể nhìn thấy toàn bộ trang, vì trang sẽ không được rendered đầy đủ cho đến khi tất cả mã JavaScript được downloaded, parsed, và được thực thi.

# NextJS Client Side Rendering

- Lợi thế của CSR
  - Với client side rendering, ta có thể tạo ra các giao diện người dùng có độ tương tác cao, khi mà JavaScript được chuyển về trình duyệt của người dùng.
  - Client side rendering có thể cung cấp hiệu suất tốt hơn trong trường hợp ta ghé thăm trang nhiều lần, nhờ cơ chế caching.
  - Giúp giảm bớt các tính toán ở phía server cho quá trình rendering views.
- Bất lợi của CSR
  - Client Side Rendering có thể tác động đến kết quả SEO của trang web. Một số các search engine crawlers có thể không thực thi JavaScript và vì thế nó chỉ có thể thấy một trạng thái trống ban đầu hoặc loading của ứng dụng.
  - Client Side Rendering dẫn đến các vấn đề hiệu suất đối với người dùng với các kết nối Internet chậm, khi mà họ cần chờ tất cả các đoạn mã JavaScript được load và thực thi trước khi họ có thể thấy toàn bộ trang.

# NextJS Client Side Rendering

- Next.js cung cấp một phương pháp lai ghép, trong đó ta sử dụng kết hợp của các cách server-side rendering, static site generation, và client-side rendering, tùy thuộc vào nhu cầu của từng trang trong ứng dụng. Trong Next.js, nếu ta khai báo 'use client' ở trên đầu trang, trang sẽ trở thành client component và nó sẽ được rendered ở phía máy khách.
- Ta có thể sử dụng các React hooks và các functions bên trong client components của Next.js

# NextJS Client Side Rendering

- Ví dụ

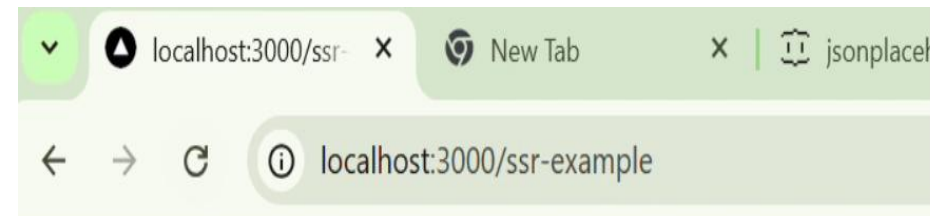
```
'use client'; // Define as Client Component

import { useState, useEffect } from 'react';

export default function CSRPage() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch('https://link/to/api');
        const json = await response.json();
        setData(json);
      } catch (error) {
        console.error('Error fetching data:', error);
      } finally {
        setLoading(false);
      }
    }
    fetchData();
  }, []);

  return (
    <div>
      <h1>{loading ? 'Loading...' : 'Client-Side Rendered Page'}</h1>
      {!loading && data && (
        <ul>
          {data.map((item) => (
            <li key={item.id}>{item.title}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```



## Header Element

## Client-Side Rendered Page

- delectus aut autem
- quis ut nam facilis et officia qui
- fugiat veniam minus
- et porro tempora
- laboriosam mollitia et enim quasi adipisci quia provident illum

# NextJS Dynamic Routing

- Dynamic routes là cơ chế được sử dụng trong các ứng dụng web, để cho phép tạo ra các routes (URLs) linh hoạt và có thể tái sử dụng dựa trên các dynamic parameters. Tính năng này là thiết yếu khi xây dựng các ứng dụng trong đó việc điều hướng các trang sẽ dựa trên các dữ liệu nhập của người dùng, hoặc các bản ghi trong CSDL, hoặc dựa trên các dữ liệu động.
- Trong Next.js, ta có thể cài đặt dynamic routing bằng cách sử dụng tên thư mục biến đổi

- Ví dụ

```
app/  
posts/  
  [id]/  
    page.jsx
```

- Sau đó, [id] có thể trở thành một dynamic route có thể match với các path như:
  - /posts/1
  - /posts/abc

# NextJS Dynamic Routing

- Ví dụ dynamic routing

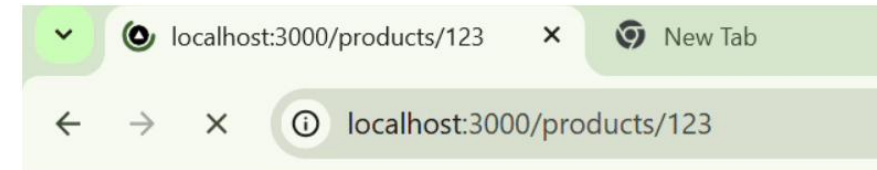
```
// app/products/[id]/page.tsx

"use client"
import { useParams } from 'next/navigation';

export default function ProductPage() {
  const { id } = useParams();

  return (
    <div>
      <h1>Product {id}</h1>
      <p>This is the product page for item {id}.</p>
    </div>
  );
}
```

- Kết quả



## Header Element

Header will remain here, when user is navigating

## Product 123

This is the product page for item 123.

# NextJS Parallel Routing

- Parallel Routing là một routing method, trong đó ta có thể tạo ra một hoặc nhiều trang đồng thời hoặc theo điều kiện, các trang này có thể có cùng layout. Cơ chế này hữu ích đối với các dynamic sections của app, chẳng hạn như các dashboards hoặc các vùng feeds trên các trang mạng xã hội.
- VD ta cần tạo một website trong đó có hiển thị thông tin người dùng, phần tin tức hiển thị bên trái, phần profile hiển thị bên phải màn hình với cấu trúc như sau.

```
app/  
  layout.tsx  
  page.tsx  
  @sidebar/  
    layout.tsx  
    page.tsx  
  @content/  
    layout.tsx  
    page.tsx
```

# NextJS Parallel Routing

- Các bước để thiết lập Parallel Routing
  - Bước 1: Enable App Router
    - Parallel Routing chỉ được hỗ trợ trong Next.js App Router, do đó ta cần tạo project với Next.js 13 trở lên.
  - Bước 2: Tạo các Named Slots
    - Parallel routes được tạo ra bằng cách sử dụng các named slots. Các slots được định nghĩa với quy ước @folder.

```
app/  
  layout.tsx  
  page.tsx  
  @sidebar/  
    layout.tsx  
    page.tsx  
  @content/  
    layout.tsx  
    page.tsx
```

# NextJS Parallel Routing

- Các bước để thiết lập Parallel Routing
  - Bước 3: Định nghĩa file layout
    - Bước tiếp theo ta cần định nghĩa file layout. File layout root là file chứa bố cục sẽ được sử dụng bởi tất cả các trang trong ứng dụng. Layout sẽ được định nghĩa trong file app/layout.tsx
  - Ví dụ

```
export default function RootLayout({
  children, sidebar, content,
}): {
  children: React.ReactNode;
  sidebar: React.ReactNode;
  content: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <div style={{ display: "flex" }}>
          <aside style={{ width: "20%", background: "#f4f4f4" }}>
            {sidebar} {/* Sidebar slot */}
          </aside>
          <main style={{ width: "80%" }}>
            {content} {/* Main Content slot */}
          </main>
        </div>
      </body>
    </html>
  );
}
```

# NextJS Parallel Routing

- Các bước để thiết lập Parallel Routing
  - Bước 4: Định nghĩa Parallel Route Segments
    - Sau khi đã thiết lập root layout, ta sẽ tạo ra lần lượt từng parallel route segments. Mỗi một parallel route segment sẽ được định nghĩa bên trong một thư mục với tên quy ước là @folder.
  - Ví dụ

```
// File: @sidebar/page.tsx

export default function Sidebar() {
  return <nav>Sidebar Content</nav>;
}

// File: @content/page.tsx

export default function Content() {
  return <section>Main Content</section>;
}
```

# NextJS Parallel Routing

- Các bước để thiết lập Parallel Routing
  - Bước 5: Định nghĩa Default Page
    - Cuối cùng ta sẽ cần định nghĩa default home page. Đây là trang sẽ được sinh ra khi người dùng duyệt root URL của ứng dụng.
  - Ví dụ:

```
// File: app/page.tsx

export default function Home() {
  return <h1>Welcome to My Page</h1>;
}
```

- Kết quả:



- Một số hàm mới trong cơ chế Data Fetching của Next.js
  - Đây là một số hàm mới được bổ sung trong Next.js để kiểm soát các quá trình cache, revalidate, request context và render flow tốt hơn.
  - Fetch mặc định (SSG)
    - Cho phép cache vĩnh viễn
    - Hỗ trợ render tĩnh
    - Ví dụ

```
await fetch("https://api.example.com/posts")
```

- Fetch không cache
  - Dữ liệu sẽ được fetch với mỗi request
  - Đảm bảo dữ liệu luôn mới
  - Ví dụ

```
await fetch(url, { cache: "no-store" })
```

# NextJS

- Fetch với revalidate (ISR)
  - Dữ liệu sẽ được cập nhật sau khoảng thời gian
  - Được sử dụng phổ biến
  - Ví dụ

```
await fetch(url, {  
  next: { revalidate: 60 },  
})
```

- revalidatePath()
  - Cho phép revalidate cache theo url
  - Được sử dụng khi submit form hoặc khi update/delete dữ liệu
  - Dùng với Server Action
  - Ví dụ

```
import { revalidatePath } from "next/cache"  
  
revalidatePath("/posts")
```

# NextJS

- cache() – memorize hàm server
  - Cho phép cache kết quả của hàm async, không chỉ fetch
  - Giúp tránh phải truy vấn database nhiều lần
  - Cho phép cache theo request/build
  - Ví dụ

```
import { cache } from "react"

const getPosts = cache(async () => {
  return db.post.findMany()
})
```

- unstable\_cache()
  - Cho phép cache hàm server có revalidate & tag
  - Được sử dụng khi fetch database trực tiếp
  - Dùng khi ta không sử dụng fetch
  - Ví dụ

```
import { unstable_cache } from "next/cache"

const getPosts = unstable_cache(
  async () => db.post.findMany(),
  ["posts"],
  { revalidate: 60 }
)
```

# NextJS

- headers()
  - Cho phép lấy HTTP headers của request (Server Component)
  - Được sử dụng cho quá trình device detection
  - Dùng cho quá trình xác thực và A/B testing
  - Ví dụ

```
import { headers } from "next/headers"

const headersList = headers()
const userAgent = headersList.get("user-agent")
```

- cookies()
  - Cho phép đọc/ghi cookie trên server
  - Được sử dụng cho quá trình xác thực, session và preference
  - Ví dụ

```
import { cookies } from "next/headers"

const cookieStore = cookies()
const token = cookieStore.get("token")
```

# NextJS

- notFound()
  - Cho phép dừng render và trả về trang 404
  - Không cần dùng if và không cần trả về JSX
  - Ví dụ

```
import { notFound } from "next/navigation"

if (!post) notFound()
```

- redirect()
  - Cho phép redirect ngay trên server
  - Được sử dụng cho quá trình xác thực, hoặc form đã submit thành công
  - Ví dụ

```
import { redirect } from "next/navigation"

redirect("/login")
```

# NextJS

- use()
  - Cho phép resolve Promise trực tiếp trong Component
  - Được sử dụng kết hợp với Streaming hoặc Suspense
  - Ví dụ

```
import { use } from "react"

const data = use(fetchData())
```

- draftMode()
  - Cho phép bật chế độ preview cho CMS
  - Được sử dụng cho việc preview bài viết, hoặc Headless CMS (Sanity, Contentful)
  - Ví dụ

```
import { draftMode } from "next/headers"

const { isEnabled } = draftMode()
```

# NextJS Lab

- Thực hành luyện tập với NextJS

# THANKS FOR WATCHING



4<sup>th</sup> floor – Hoa Binh tower, 106 Hoang Quoc Viet, Hanoi, Vietnam



(+84) 366 496 399



[sales.en@luvina.net](mailto:sales.en@luvina.net)



[www.luvina.net](http://www.luvina.net)